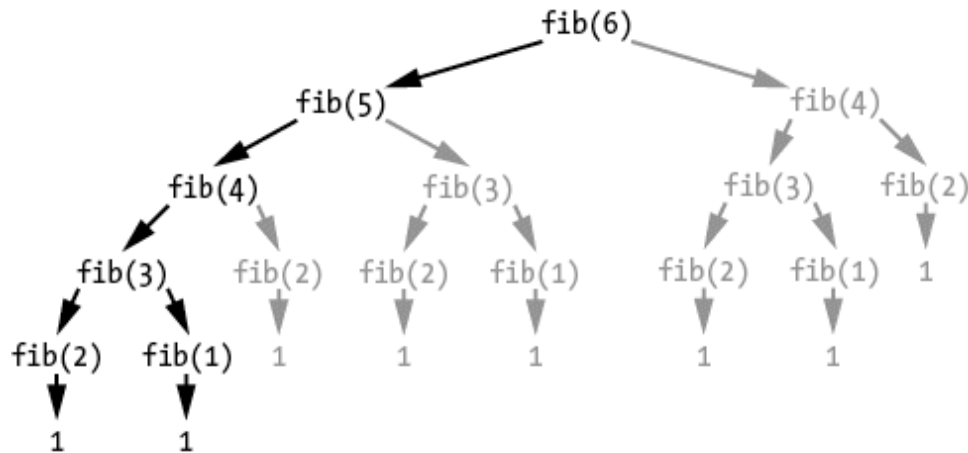


[Figure 7-1](#) shows a tree diagram of all the recursive calls, including the redundant function calls that memoization can optimize. Meanwhile, quicksort and merge sort are recursive divide-and-conquer algorithms, but their subproblems do not overlap; they are unique. Dynamic programming techniques aren't applied to these sorting algorithms.



[Figure 7-1](#): A tree diagram of the recursive function calls made starting with `fibonacci(6)`. The redundant function calls are in gray.

One approach in dynamic programming is to memoize the recursive function so that previous calculations are remembered for future function calls. Overlapping subproblems become trivial if we can reuse previous return values.

Using recursion with memoization is called *top-down dynamic programming*. This process takes a large problem and divides it into smaller overlapping subproblems. A contrasting technique, *bottom-up dynamic programming*, starts with the smaller subproblems (often related to the base case) and “builds up” to the solution of the original, large problem. The iterative Fibonacci algorithm, which begins with the base cases of the first and second Fibonacci numbers, is an example of bottom-up dynamic programming. Bottom-up approaches don’t use recursive functions.

Note that there is no such thing as top-down recursion or bottom-up recursion. These are commonly used but incorrect terms. All recursion is already top-down, so *top-down recursion* is redundant. And no bottom-up approach uses recursion, so there’s no such thing as *bottom-up recursion*.